Principles of Machine Learning

Mariette Schönfeld (session 1,3,5) *mariette.schoenfeld@kuleuven.be*
Andrea Mirenda session (2,4) *andrea.mirenda@kuleuven.be*
Isidoro Tamassia (session 6,7) *isidoro.tamassia@kuleuven.be*

Fall, 2026

Exercise Session 2: Evaluation, Methodology, and Ensembles

2.1 Evaluation Metrics and Confusion Matrices

Here is an overview of the evaluation metrics as discussed in the methodology lecture:

		predicted		
		+	-	
actual	+	TP	FN	TPR = $\frac{\text{TP}}{\text{TP+FN}}$ = recall = sensitivity
	-	FP	TN	FPR = $\frac{\text{FP}}{\text{FP+TN}}$
		Precision = $\frac{\text{TP}}{\text{TP+FP}}$		

a. Give an intuitive interpretation for precision, recall, and false positive rate (FPR).

b. Table (a) below contains 10 instances with their true labels y, and their predicted probabilities of belonging to the positive class $\hat{p}(y = 1)$.

(a)

i	y	$\hat{p}(y = 1)$
0	0	0.11
1	0	0.56
2	1	0.45
3	0	0.01
4	0	0.09
5	1	0.86
6	1	0.95
7	0	0.13
8	1	0.79
9	0	0.51

(b)

i	y	$\hat{p}(y = 1)$	$\hat{y}_{0.5}$	$\hat{y}_{0.3}$	$\hat{y}_{0.7}$
3	0	0.01	0		
4	0	0.09	0		
0	0	0.11	0		
7	0	0.13	0		
2	1	0.45	0		
9	0	0.51	1		
1	0	0.56	1		
8	1	0.79	1		
5	1	0.86	1		
6	1	0.95	1		

Construct the confusion matrix for probability threshold 0.5, i.e., for

$$\hat{y}_{0.5} = [0, 1, 0, 0, 0, 1, 1, 0, 1, 1].$$

c. Compute the accuracy and the error for the results in table (a) above. Write the accuracy in terms of TP, FN, FP, and TN.

d. Draw the ROC curve for thresholds 0.3, 0.5, and 0.7. Table (b) already has the instances from table (a) sorted by predicted probability value. The horizontal lines correspond to the three thresholds (that is, you only need to construct three confusion matrices). Interpret these values. Which classifier would you prefer? Are any of these three points in the ROC curve suboptimal?

e. Draw the precision-recall curve for the same points.

2.2 Programming Exercise: K-Fold Cross Validation

You can find a Python script `kfold.py` on Toledo. This script will guide you through the following questions:

- Which model (parameters) should you pick and why?
- Which model of the two plotted should you pick and why?

Answer the questions printed to the standard output by the script. You are encouraged to experiment!

2.3 Boosted Decision Trees

In this assignment, you will implement a Boosted Decision Tree algorithm from scratch and apply it to the Ionosphere dataset to classify radar signals as either "good" or "bad." You will write a class in Python that implements the core functionality of the boosting algorithm, and then run it on the Ionosphere dataset to evaluate its performance.

Instructions:

1. **Download the dataset.** A CSV version of this dataset has been provided in the assignment materials. Alternatively, The dataset is available from the UCI Machine Learning Repository: Ionosphere Dataset.
2. **Implement the Boosted Decision Tree class.** Your task is to complete the BoostedDecisionTree class that implements the following key components:
    - Information Gain Calculation (info_gain);
    - Entropy Calculation (entropy);
    - Finding the Best Attribute to Split (find_best_attrib);
    - Weight Update Mechanism (update_weights);
    - Boosting Algorithm (fit);
    - Prediction Method (predict).
3. **Steps to follow.**
    - The skeleton of the BoostedDecisionTree class has been provided in the notebook;
    - You will need to fill in the missing code sections (these are marked with comments);
    - Your implementation should be able to handle boosting over multiple iterations (i.e., multiple decision stumps);
    - After training the model, you should be able to use the predict method to classify new radar signals.
4. **Testing your model.**
    - Split the dataset into training and testing sets using an 80/20 split;
    - Train your BoostedDecisionTree model on the training data;

- Make predictions on the test data and evaluate the model's performance using accuracy as the metric;
- Optionally, you can also visualize the error reduction across the boosting iterations.

2.4 Combining Classifications

In the table below, some instances are given and the predicted probabilities of it belonging to the positive class, as predicted by different binary classifiers. The true class of the instance is also listed. We will now look at some methods to combine the classifiers for an ensemble prediction.

The table shows the predicted probability of classifier C for instance I to be positive, alongside the true instance class T.

	C_1	C_2	C_3	T
I_1	0.3	0.8	0.7	$+$
I_2	0.4	0.4	0.9	$+$
I_3	0.2	0.6	0.4	$-$

a. Assume a classifier predicts the positive class if the probability is higher than $1/2$. Calculate the majority vote for the ensemble prediction giving each classifier an equal weight.

b. Now calculate a weighted vote prediction. Use as weights $p - 1/2$, with p the predicted probability for the instance to be positive. This quantity measures the certainty of the prediction.

c. Use a Bayesian estimate where the probability for each classifier is given by its accuracy, normalized over all the accuracies.

What are some noticeable results about the different ways to predict in terms of accuracy?